

Unit-1: Introduction

Software:

- Software is a **collection of programs, associated documentation and configuration data** which is needed to make these programs operate correctly.
- **Types of Software:**
 - **System Software:** Manages hardware and provides basic services (e.g., Operating Systems like Windows, Linux).
 - **Application Software:** Helps users do specific tasks (e.g., MS Word, web browsers).
 - **Embedded Software:** Built into devices like cars, washing machines, or IoT devices.
 - **Web and Mobile Apps:** Modern interactive applications that run on browsers or smartphones.
 - **Middleware:** Connects different software or systems (e.g. JDBC, Kafka).

Program Vs. Software Product:

Program	Software Product
Are developed by individuals for their personal use.	Are usually developed by a group of software engineers and have multiple users.
Small in size and have limited functionality	Medium or large size program and have complex functionality.
Usually do not have good UI and lack proper documentation	Not only consists of program code but also all the associated documents (like SRS, design Document, test documents and user's manuals)

Characteristics of Software: “What software is by nature.”

- **Intangible:** Can't be touched physically.
- **Developed, not manufactured:** Built through coding, not assembled like a machine.
- **Doesn't wear out but does deteriorate:** Hardware degrades, but software only “fails” when it has bugs or becomes outdated.
- **Customizable:** Can be updated or changed to meet new requirements.

Attributes of Good Software: “How well the software does its job.”

- **Correct:** Works according to requirements.
- **Reliable:** Consistently produces correct results.
- **Efficient:** Uses minimal resources (CPU, memory, network).
- **Maintainable:** Easy to update or fix.
- **Usable:** Easy for users to operate.
- **Portable:** Can be transferred from one environment to another, without or minimum changes.
- **Secure:** Protects data and resists attacks.

Constructing a bridge Vs Writing a program:

Writing a Program	Constructing a bridge
Requirements typically change during all phases of development	Requirement typically do not change much
Progress is not immediately visible	Progress being made can actually be seen
Ownership can be copied easily	Ownership of physical projects is harder to replicate
Productivity is often affected by employee change	Productivity is not greatly affected by employee change
Easily Customized	Requires Effort to Customized

Software Myths:

- **Management Perspectives:**
 - Having the latest computers and tools automatically guarantees high-quality software.
 - Software is easy to change
 - Adding more people to a late project will finish it faster.
- **Customer Perspectives:**
 - A general statement of objectives is enough to start coding.
 - Requirements can easily be changed later with no impact.
 - A working program is all that is needed (no documentation/testing).
 - Low cost and short schedule can still yield high quality software.

➤ **Developer Perspectives:**

- Once the software is demonstrated, the job is done.
- Software quality can't be assessed before testing.
- Aim is to develop working programs.
- Only deliverable for a software development project is the tested code.

Software Engineering:

- Software engineering is a branch of engineering concerned with the development of software products using well-defined scientific principles, methods, and procedures.
- The goal of software engineering is to produce efficient and reliable software products.
- **Software Engineering Vs Computer Science:**

Software engineering is about making useful programs, while computer science is about understanding how computers work.

- **Software Engineering Vs System Engineering:**

Software engineering develops and maintains software applications; system engineering designs and manages the entire system, including software, hardware, and users.

Generic Phases of Software Engineering:

- **Definition Phase (Requirements/Planning Phase):**

- Following activities are performed:
 - **Requirement Gathering:** Collect needs from all stakeholders.
 - **Feasibility Study:** Check technical, economic, and operational feasibility.
 - **Requirement Analysis:** Examine, validate and prioritize collected requirements.
 - **Project Planning:** Define scope, schedule, resources, and budget.
 - **Documentation:** Document detailed functional and non-functional requirements.

- **Development Phase (Design & Implementation Phase):**

- Following activities are performed:
 - **System Design:** Create architecture and detailed design of the software.
 - **Coding/Implementation:** Write the actual program code.
 - **Testing:** Evaluating software to find defects and ensure it meets requirements.
 - **Documentation:** Prepare user manuals and technical documents.

➤ **Support Phase (Maintenance Phase):**

- Ongoing activities to keep software running smoothly, including maintenance, updates, and user assistance.
- **4 types of maintenance:**
 - **Corrective Maintenance:**
Fixing errors or bugs discovered after the software is in use.
 - **Adaptive Maintenance:**
Modifying the software to work in a changed environment.
 - **Perfective Maintenance:**
Improves performance or adds new features.
 - **Preventive Maintenance:**
Prevents future problems by improving code or design.

Software Engineering Costs:

- The total money and resources needed to build, test, run, and maintain a software.
- Different types of costs involved in software engineering:
 - **Development Cost:** Money spent on designing, coding, and building the software.
 - **Testing Cost:** Cost of checking the software for errors and ensuring quality.
 - **Maintenance Cost:** Fixing bugs, updating, and improving software after release.
 - **Training Cost:** Teaching users or staff how to use the software.
 - **People Cost:** Salaries of developers, testers, project managers, and support staff.
 - **Hardware/Software Tools Cost:** Buying servers, computers, or special tools needed to develop and run the software.
 - **Documentation Cost:** Money spent to **write manuals, guides, and technical documents** for users or developers.

Key Challenges that Software Engineering Facing:

- **Changing Requirements:** Clients often change their needs during development, making it hard to keep up.
- **Complexity of Systems:** Modern software can be very large and complex, making it tough to design, test, and maintain.
- **Time and Budget Limits:** Projects often have tight deadlines and limited money, which puts pressure on developers.
- **Security Risks:** Protecting software from hackers and cyberattacks is a constant challenge.

- **Technology Changes Fast:** New tools, languages, and platforms come quickly, so engineers must keep learning.
- **Quality & Reliability** – Ensuring the software is bug-free, secure, and performs well.

Professional Practices in Software Engineering:

- **Following Standards & Guidelines:** Writing code and documents according to accepted rules and best practices (e.g. ISO 9000, 9001).
- **Using Version Control:** Keeping track of code changes using tools like Git.
- **Proper Documentation:** Properly writing manuals, guides, and technical documents.
- **Testing & Quality Assurance:** Systematically checking software for errors and reliability.
- **Effective Communication and Collaboration:** Ensure teams work efficiently, make informed decisions, and deliver high-quality software reliably.
- **Consideration of economic/social impact:** Software solutions are responsible, sustainable, and beneficial to society while being cost-effective.
- **Ethical Practices:** Following ethical standards, like honesty, data privacy, and intellectual property respect.